Implement Client Script & UI Policy (Incident)

ServiceNow Micro Project Report & Technical Implementation Guide

Team ID:	SWTID-2026-5957
Team Size:	3 Members
Team Leader:	**RIFANA FIRDOUS J**
	Gmail: ayifirmah356@gmail.com
Team Member 1:	**SIVARANJANI S**
	Gmail: ssivaranjanissivaranjani27@gmail.com
Team Member 2:	**SIVA SHREE S**
	Gmail: shreesiva260@gmail.com

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted [cite: 1].

Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform [cite: 1].

Key Skills Applied

- Incident Management & Form Configuration [cite: 1]
- UI Policy & UI Policy Actions Configuration [cite: 1]
- Client Scripts Development (onChange, onSubmit, onCellEdit) [cite: 1]
- Form Validation & Error Handling [cite: 1]

Task 1: Create UI Policy on Incident

Created a UI Policy named **High Impact Control** targeting the Incident table. The policy applies when `Impact = 1 - High` with `Reverse if true` and `On load` enabled. A UI policy action is configured for the **Assignment group** field to set it as mandatory [cite: 1].

Task 2: Create UI Policy Action - Urgency

Configured an additional UI Policy Action under the High Impact Control policy targeting the **Urgency** field to make it **Read-only** whenever the Incident impact is set to High, preventing manual overrides while keeping the field visible [cite: 1].

Task 3: Create onChange Client Script

Created an `onChange` Client Script named **Auto set urgency for high impact** on the Incident table, triggered by the `Impact` field. When Impact is changed to High (`1`), the script automatically updates Urgency to High and displays an information message [cite: 1].

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

Task 4: Create onSubmit Client Script (Save Validation)

Created an `onSubmit` Client Script named **Prevent save if assigned to missing**. This script ensures that if an Incident has High impact, the `Assigned to` field cannot be left blank, blocking record submission and displaying an error message if violated [cite: 1].

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {
        g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact
incidents.');
        return false;
    }
    return true;
}
```

Task 5: Create onCellEdit Client Script

Created an `onCellEdit` Client Script named **Prevent state change via list edit** on the Incident table for the `State` field to block direct list-view modifications, instructing users to open the form [cite: 1].

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be changed from the list. Please open the Incident.');
    callback(false);
}
```

Task 6: Testing & Verification

Comprehensive testing was conducted across all configured rules [cite: 1]:

- **Mandatory Enforcement:** Verified that saving a High Impact incident without an Assigned To value triggers an error banner and blocks creation [cite: 1].
- **Successful Save:** Confirmed successful submission once mandatory requirements are met, with automated urgency adjustments functioning seamlessly [cite: 1].
- **Reverse Condition:** Tested changing Impact from High to Medium, confirming that read-only status and mandatory fields revert correctly [cite: 1].
- **List Edit & Form Updates:** Verified that list-view state changes are successfully blocked by cell-edit scripts, while form-based updates proceed normally [cite: 1].

Conclusion

> The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms, serving as a reliable benchmark for ServiceNow platform administration [cite: 1].